

Week 14: Learning to Act, Respecting Physics and Engineering Responsibly

Artificial Intelligence Applications in Engineering (MUH-920 Mühendislikte Yapay Zeka Uygulamaları). Lecture notes

Prof. Dr. Utku Kose

Full Professor, Department of Computer Engineering, Süleyman Demirel University, Isparta, Türkiye
Founding Director, AI Application and Research Center (YAZEM), Süleyman Demirel University
Head of the Computer Science Division, Department of Computer Engineering, Süleyman Demirel University

Additional affiliations: University of North Dakota (USA), Universidad Panamericana (Mexico City, Mexico), Vel Tech University (Chennai, India)

IEEE Senior Member, ACM Professional Member

ORCID 0000-0002-9652-6415 | utkukose.com | github.com/utkukose

utkukose@sdu.edu.tr | utku.kose@und.edu | ukose@up.edu.mx | utkukose@gmail.com

Week overview, study path and tasks: <https://github.com/utkukose/ai-applications-in-engineering-NB-lecture/blob/main/weeks/week-14/README.md>

Lecture page with animations and knowledge checks: <https://utkukose.github.io/ai-applications-in-engineering-NB-lecture/weeks/week-14/lecture.html>

Interactive lab with self-assessment and reflection: <https://utkukose.github.io/ai-applications-in-engineering-NB-lecture/weeks/week-14/lab.html>

Colab notebook with Python step 14 and the hands-on work:

https://colab.research.google.com/github/utkukose/ai-applications-in-engineering-NB-lecture/blob/main/weeks/week-14/NB14_rl_pinn_digital_twins.ipynb

Content licensed under CC BY 4.0. This course is updated in line with current developments in the field.

Last update: October 2026.

Overview

The course began with a thermostat that followed fixed rules. It ends with agents that learn their own rules from reward, with models that obey physical laws while learning from data, and with the responsibilities that come with deploying such systems. Reinforcement learning learns a policy by trial and error [1, 2]; physics-informed neural networks add governing equations to the loss of a network [3, 4]; and digital twins keep a model synchronised with a physical asset [5, 6]. The week closes with safety, standards and regulation, including the European Union's Artificial Intelligence Act, the NIST AI Risk Management Framework and ISO/IEC 42001 [7, 8, 9], and with guidance for the final project.

Estimated study time: 10 to 12 hours.

Final project. The final project is due at the end of this week. The weekly task of this week is a last check of reproducibility and of the report before submission, following the [project page](#).

Learning outcomes

By the end of the week, students are expected to formulate a control task as a Markov decision process, to apply the Q-learning update and an epsilon-greedy policy, to explain why exploration on physical systems needs simulators and safeguards, to train a physics-informed network for an ordinary differential equation and compare it with a data-only network, to identify a simple model from data as the core of a digital twin, and to place an engineering AI system within the risk categories of the EU AI Act and the functions of the NIST framework.

Python in this week

These notes close with Python step 14: Environments, learning and organised code. It covers functions with type hints, parameters in dataclasses, an environment class with the interface of Gymnasium, tabular Q-learning, a physics residual with automatic differentiation, tests with assert, modules and records of runs, applied to the heating task that closes the course [1, 10, 11]. In the course materials, the step is taught at the start of the Colab notebook, where each snippet runs in a cell next to its explanation and short exercises give immediate feedback, and later sections of the notebook mark further constructs as Python moves.

Learning to act

In Week 1, an engineer wrote the rule of the thermostat. Reinforcement learning lets an agent find the rule itself. At every step the agent observes a state, chooses an action and receives a reward and a new state; its goal is a policy that maximises the expected sum of discounted future rewards [1]. The formal model is a Markov decision process: The next state depends only on the current state and action. The discount factor gamma, between 0 and 1, sets how far the agent looks ahead. Unlike supervised learning, nobody tells the agent the correct action; it must explore to find out which actions pay off, and exploit what it has learned to collect reward.

The value of taking action a in state s and acting well afterwards is $Q(s, a)$. Q-learning improves an estimate of Q after every step with $Q(s, a) = Q(s, a) + \alpha (r + \gamma \max_{a'} Q(s', a') - Q(s, a))$, where alpha is a learning rate and the maximum runs over the actions in the next state; Watkins and Dayan proved that it converges to the optimal values under suitable conditions [2]. An epsilon-greedy policy explores with a random action with probability epsilon and exploits the best known action otherwise. Tables work for small discrete problems; for large or continuous state spaces, a neural network approximates Q, as in the deep Q-network that learned to play Atari games from pixels [12].

Animation: Q-learning on a cliff walk. The agent starts at the lower left and must reach the lower right without stepping into the cliff, which costs heavily. Each step costs a little. Watch the arrows of the greedy policy and the colours of the state values change as episodes accumulate. Raise epsilon to explore more. Run it on the [lecture page](#).

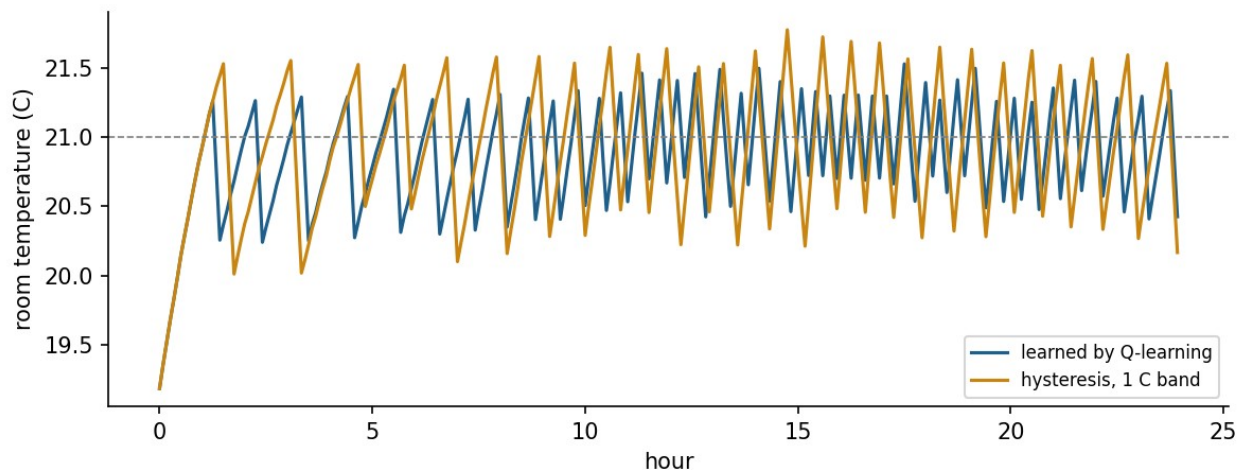


Figure 14.1. Room temperature under the hysteresis rule of Week 1 and under a policy learned by Q-learning on the same synthetic winter day. The learned policy discovers a switching band by itself.

Check your understanding. $Q(s, a) = 2.0$, the reward is -1, gamma = 0.9, the best Q-value in the next state is 3.0 and alpha = 0.5. What is the updated $Q(s, a)$?

- A. 1.85
- B. 2.0
- C. 1.7
- D. 3.7

Answer: A. The target is $-1 + 0.9 \times 3.0 = 1.7$; the update gives $2.0 + 0.5 \times (1.7 - 2.0) = 1.85$.

Formulas: Returns, values and the Bellman equations

A Markov decision process consists of states s , actions a , transition probabilities $P(s' | s, a)$ of the next state s' , rewards r and a discount factor γ with $0 \leq \gamma < 1$ [1]. From the time t on, the agent collects the rewards $r_{t+1}, r_{t+2}, \dots$, and its return is their discounted sum:

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

The reward r_{t+k+1} counts with the weight γ^k . A constant reward r in every step is worth $r + \gamma r + \gamma^2 r + \dots = r/(1 - \gamma)$, the sum of a geometric series, so with $\gamma = 0.9$ a reward of 1 per step is worth 10, and $1/(1 - \gamma)$ acts as the horizon of the agent.

A policy π chooses the actions. The value of a state is the expected return of an agent that starts there and follows π , and the action value adds a first action a :

$$V_{\pi}(s) = E_{\pi}[G_t | s_t = s], Q_{\pi}(s, a) = E_{\pi}[G_t | s_t = s, a_t = a]$$

The best possible values satisfy the Bellman optimality equations: The value of a state is the best action value, and an action value is the expected sum of the reward and the discounted value of the next state [1]:

$$V(s) = \max_a Q(s, a), Q(s, a) = \sum_{s'} P(s' | s, a) (r(s, a, s') + \gamma V(s'))$$

Value iteration starts from $V_0(s) = 0$ and turns these equations into an update that is repeated for all states, one sweep after another:

$$V_{k+1}(s) = \max_a \sum_{s'} P(s' | s, a) (r(s, a, s') + \gamma V_k(s'))$$

Each sweep shrinks the largest error of the values at least by the factor γ , so the values converge, and the greedy policy, which picks an action with the largest action value, becomes optimal.

Q-learning learns action values from experience, without knowing the transition probabilities [2]. After each transition from s with the action a to s' with the reward r , the estimate moves by the learning rate α towards the target $r + \gamma \max_{a'} Q(s', a')$:

$$\delta = r + \gamma \max_{a'} Q(s', a') - Q(s, a), Q(s, a) \leftarrow Q(s, a) + \alpha \delta$$

The difference δ is the temporal-difference error. An ϵ -greedy policy with m actions chooses a random action with the probability ϵ , so the greedy action has the probability $1 - \epsilon + \epsilon/m$ and every other action ϵ/m .

Worked example: The return of a maintenance history

A pump is inspected at the start of every week and is good (G), worn (W) or failed (F). The engineer can continue (c) or service (s). Continuing earns 10 units of production with a good pump, 6 with a worn pump and nothing with a failed one. Servicing costs 2, 4 or 20 units in the three states and returns the pump to G for the next week. A good pump wears within a week with the probability 0.3, and a worn pump fails with the probability 0.4.

In one history, the engineer continued three times and then serviced the worn pump, with the rewards 10, 10, 6 and -4 . With $\gamma = 0.9$ the weights are $0.9^0 = 1$, $0.9^1 = 0.9$, $0.9^2 = 0.81$ and $0.9^3 = 0.729$:

$$G_0 = 10 + 0.9 \times 10 + 0.81 \times 6 + 0.729 \times (-4) = 10 + 9 + 4.86 - 2.916 = 20.944$$

The service at the end lowers the return of this history, but it restores the production of later weeks, which a return of four weeks does not see. The values of the next example weigh all future weeks.

Worked example: Value iteration for pump maintenance

Step 1. The first sweep. With $V_0 = 0$ in every state, every action value equals its reward, and the larger one is the new value:

$$V_1(G) = \max(10, -2) = 10, V_1(W) = \max(6, -4) = 6, V_1(F) = \max(0, -20) = 0$$

Step 2. The second sweep. The action values weigh the values V_1 of the possible next states with their probabilities:

$$Q_2(G, c) = 10 + 0.9 \times (0.7 \times 10 + 0.3 \times 6) = 10 + 0.9 \times (7 + 1.8)$$

$$Q_2(G, c) = 10 + 0.9 \times 8.8 = 10 + 7.92 = 17.92$$

$$Q_2(W, c) = 6 + 0.9 \times (0.6 \times 6 + 0.4 \times 0) = 6 + 0.9 \times (3.6 + 0)$$

$$Q_2(W, c) = 6 + 0.9 \times 3.6 = 6 + 3.24 = 9.24$$

Servicing and continuing with a failed pump lead to a single next state, and the value of a state is its larger action value:

State	Continue, c	Service, s	V_2 , the larger
G	17.92	$-2 + 0.9 \times 10 = -2 + 9 = 7$	17.92
W	9.24	$-4 + 0.9 \times 10 = -4 + 9 = 5$	9.24
F	$0 + 0.9 \times 0 = 0 + 0 = 0$	$-20 + 0.9 \times 10 = -20 + 9 = -11$	0

Step 3. The third sweep. With the values V_2 :

$$Q_3(G, c) = 10 + 0.9 \times (0.7 \times 17.92 + 0.3 \times 9.24) = 10 + 0.9 \times (12.544 + 2.772)$$

$$Q_3(G, c) = 10 + 0.9 \times 15.316 = 10 + 13.7844 = 23.7844$$

$$Q_3(W, c) = 6 + 0.9 \times (0.6 \times 9.24 + 0.4 \times 0) = 6 + 0.9 \times (5.544 + 0)$$

$$Q_3(W, c) = 6 + 0.9 \times 5.544 = 6 + 4.9896 = 10.9896$$

Servicing and continuing with a failed pump lead to a single next state, and the value of a state is its larger action value:

State	Continue, c	Service, s	V_3 , the larger
G	23.7844	$-2 + 0.9 \times 17.92 = -2 + 16.128 = 14.128$	23.7844
W	10.9896	$-4 + 0.9 \times 17.92 = -4 + 16.128 = 12.128$	12.128
F	$0 + 0.9 \times 0 = 0 + 0 = 0$	$-20 + 0.9 \times 17.92 = -20 + 16.128 = -3.872$	0

Servicing a worn pump now beats continuing, because the value of a good pump, which a service restores, has grown faster than the value of continuing with a worn one.

Step 4. The exact values. Further sweeps change the values less and less, and from the fourth sweep on the greedy policy stays the same: Continue in G, service in W and service in F. The values of this policy solve its three Bellman equations, which have no maximum because the actions are fixed:

$$V(G) = 10 + 0.9 \times (0.7 V(G) + 0.3 V(W))$$

$$V(W) = -4 + 0.9V(G), V(F) = -20 + 0.9V(G)$$

With $0.9 \times 0.7 = 0.63$ and $0.9 \times 0.3 = 0.27$, inserting the second equation into the first gives

$$V(G) = 10 + 0.63V(G) + 0.27 \times (-4 + 0.9V(G))$$

$$V(G) = 10 + 0.63V(G) - 1.08 + 0.243V(G) = 8.92 + 0.873V(G)$$

$$V(G) = \frac{8.92}{1 - 0.873} = \frac{8.92}{0.127} = 70.2362$$

$$V(W) = -4 + 0.9 \times 70.2362 = -4 + 63.2126 = 59.2126, V(F) = -20 + 63.2126 = 43.2126$$

Step 5. The check. The policy is optimal because no other action does better with these values. Continuing with a worn pump, continuing with a failed pump and servicing a good pump give

$$Q(W, c) = 6 + 0.9 \times (0.6 \times 59.2126 + 0.4 \times 43.2126) = 6 + 0.9 \times (35.5276 + 17.285)$$

$$Q(W, c) = 6 + 0.9 \times 52.8126 = 6 + 47.5313 = 53.5313$$

$$Q(F, c) = 0 + 0.9 \times 43.2126 = 0 + 38.8913 = 38.8913$$

$$Q(G, s) = -2 + 0.9 \times 70.2362 = -2 + 63.2126 = 61.2126$$

which are all smaller than $V(W) = 59.2126$, $V(F) = 43.2126$ and $V(G) = 70.2362$. The high cost of a failure makes preventive service of a worn pump worthwhile, the logic of condition-based maintenance. Part D of the [interactive lab](#) runs the sweeps for any rewards, probabilities and discount factors.

Worked example: Q-learning from five transitions

An agent that does not know the probabilities of the pump starts with $Q = 0$ for every state and action, uses $\alpha = 0.5$ and $\gamma = 0.9$, and observes five transitions.

Update 1. In state G, the action continue gave the reward 10 and the next state G, whose larger estimate is $\max(0, 0) = 0$:

$$\delta = 10 + 0.9 \times 0 - 0 = 10 + 0 - 0 = 10 - 0 = 10$$

$$Q(G, c) = 0 + 0.5 \times 10 = 0 + 5 = 5$$

Update 2. In state G, the action continue gave the reward 10 and the next state W, whose larger estimate is $\max(0, 0) = 0$:

$$\delta = 10 + 0.9 \times 0 - 5 = 10 + 0 - 5 = 10 - 5 = 5$$

$$Q(G, c) = 5 + 0.5 \times 5 = 5 + 2.5 = 7.5$$

Update 3. In state W, the action continue gave the reward 6 and the next state F, whose larger estimate is $\max(0, 0) = 0$:

$$\delta = 6 + 0.9 \times 0 - 0 = 6 + 0 - 0 = 6 - 0 = 6$$

$$Q(W, c) = 0 + 0.5 \times 6 = 0 + 3 = 3$$

Update 4. In state F, the action service gave the reward -20 and the next state G, whose larger estimate is $\max(7.5, 0) = 7.5$:

$$\delta = -20 + 0.9 \times 7.5 - 0 = -20 + 6.75 - 0 = -13.25 - 0 = -13.25$$

$$Q(F, s) = 0 + 0.5 \times (-13.25) = 0 + (-6.625) = -6.625$$

Update 5. In state G, the action continue gave the reward 10 and the next state G, whose larger estimate is $\max(7.5, 0) = 7.5$:

$$\delta = 10 + 0.9 \times 7.5 - 7.5 = 10 + 6.75 - 7.5 = 16.75 - 7.5 = 9.25$$

$$Q(G, c) = 7.5 + 0.5 \times 9.25 = 7.5 + 4.625 = 12.125$$

After five transitions, the estimate $Q(G, c) = 12.125$ is still far below its value 70.2362 from the previous example, and $Q(F, s) = -6.625$ is lower than $Q(F, c) = 0$, so a greedy agent would leave a failed pump idle. Exploration corrects such early errors: With $\epsilon = 0.1$ and two actions, the greedy action is taken with the probability $1 - 0.1 + 0.1/2 = 0.95$ and the other action with $0.1/2 = 0.05$, so the agent keeps trying repairs. Part E of the lab runs Q-learning on the pump for thousands of weeks.

Reinforcement learning in engineering

Control problems were among the first testbeds of reinforcement learning: The cart-pole balancing task goes back to adaptive elements studied in the early 1980s [13], and Gymnasium now provides it with a standard interface for experiments [11]. Robotics has used reinforcement learning for manipulation and locomotion [14], and deep reinforcement learning has controlled the magnetic coils of a tokamak in a fusion experiment [15]. These successes share a feature: The agent learned mostly in simulation. Exploration on a real plant can damage equipment or endanger people, the reward may be satisfied in unintended ways, and a policy trained under one set of conditions may fail under others; Amodei and colleagues list safe exploration, reward hacking and distributional shift among the concrete problems of AI safety [16]. Simulators, constraints on actions, human oversight and gradual deployment are therefore part of any engineering use.

Check your understanding. Why do most engineering applications of reinforcement learning train the agent in simulation first?

- A. Simulations are always more accurate than reality
- B. Exploration on the real system can be unsafe or costly, and simulation allows many cheap trials
- C. Reinforcement learning cannot run on real hardware
- D. Rewards cannot be defined for real systems

Answer: B. Safe exploration is a central problem; simulators and digital twins provide a safe place to make mistakes [16].

Formulas: Rewards and the cross-entropy method

The reward is the specification of the task: The agent maximises exactly what it is rewarded for, and a missing or badly weighted term invites behaviour that the engineer did not intend [16]. The heating agent of the notebook receives, after it has chosen the heater command u_t and the room has reached the temperature T_{t+1} , the reward

$$r_{t+1} = -|T_{t+1} - 21| - w_s s_t - w_e u_t$$

where s_t is 1 if the command differs from the previous one and 0 otherwise. The first term rewards comfort, the second penalises switching, which wears the relay, and the third penalises energy; the notebook uses $w_s = 0.3$ and $w_e = 0.05$.

Policy search methods adjust the parameters θ of a policy directly. The cross-entropy method draws N parameter vectors from a normal distribution with the mean μ and the standard deviation σ , runs an episode with each, keeps the K vectors with the highest returns, the elite, and fits the distribution to the elite:

$$\mu_{\text{new}} = \frac{1}{K} \sum_{k=1}^K \theta_k, \sigma_{\text{new}}^2 = \frac{1}{K} \sum_{k=1}^K (\theta_k - \mu_{\text{new}})^2$$

For vectors, these formulas apply to every component. The distribution narrows around good parameters within a few iterations; a small constant added to σ , 0.05 in the notebook, keeps it from collapsing too early. The method needs no gradients, like the population methods of Weeks 5 and 6, which suits simulators that are treated as black boxes.

Worked example: How the reward weights choose a behaviour

Two heater trajectories of four steps start from a heater that was off. Every reward splits into the comfort and energy part, which does not depend on w_s , and the switching penalty $-w_s s_t$, and the discount factor is $\gamma=0.9$. Trajectory A switches in every step and stays close to 21 °C:

Step	Weight γ^{k-1}	T (°C)	u	Switch	Comfort and energy
1	1	21.2	1	1	$-0.2 - 0.05 = -0.25$
2	0.9	20.9	0	1	$-0.1 - 0 = -0.1$
3	0.81	21.1	1	1	$-0.1 - 0.05 = -0.15$
4	0.729	20.8	0	1	$-0.2 - 0 = -0.2$

Trajectory B switches twice and drifts further from 21 °C:

Step	Weight γ^{k-1}	T (°C)	u	Switch	Comfort and energy
1	1	21.4	1	1	$-0.4 - 0.05 = -0.45$
2	0.9	21.8	1	0	$-0.8 - 0.05 = -0.85$
3	0.81	21.3	0	1	$-0.3 - 0 = -0.3$
4	0.729	20.9	0	0	$-0.1 - 0 = -0.1$

Step 1. Discounted sums. The comfort and energy parts and the switches, each weighted by γ^{k-1} , add up to

$$C_A = 1 \times (-0.25) + 0.9 \times (-0.1) + 0.81 \times (-0.15) + 0.729 \times (-0.2)$$

$$C_A = -0.25 - 0.09 - 0.1215 - 0.1458 = -0.6073$$

$$C_B = 1 \times (-0.45) + 0.9 \times (-0.85) + 0.81 \times (-0.3) + 0.729 \times (-0.1)$$

$$C_B = -0.45 - 0.765 - 0.243 - 0.0729 = -1.5309$$

$$S_A = 1 + 0.9 + 0.81 + 0.729 = 3.439, S_B = 1 + 0.81 = 1.81$$

so the returns are $G_A = -0.6073 - 3.439 w_s$ and $G_B = -1.5309 - 1.81 w_s$.

Step 2. The weight of the notebook. With $w_s=0.3$:

$$G_A = -0.6073 - 3.439 \times 0.3 = -0.6073 - 1.0317 = -1.639$$

$$G_B = -1.5309 - 1.81 \times 0.3 = -1.5309 - 0.543 = -2.0739$$

Trajectory A has the higher return, so a learning agent is drawn towards frequent switching.

Step 3. A heavier penalty. With $w_s=2$:

$$G_A = -0.6073 - 3.439 \times 2 = -0.6073 - 6.878 = -7.4853$$

$$G_B = -1.5309 - 1.81 \times 2 = -1.5309 - 3.62 = -5.1509$$

Now trajectory B wins. The two returns are equal where $-0.6073 - 3.439 w_s = -1.5309 - 1.81 w_s$, that is at

$$w_s = \frac{-0.6073 - (-1.5309)}{3.439 - 1.81} = \frac{0.9236}{1.629} = 0.567$$

The weight is an engineering decision about relay wear against comfort, and the agent only carries it out. Part F of the [interactive lab](#) compares the two trajectories for any weights.

Worked example: One iteration of the cross-entropy method

A controller gain θ is tuned on a simulator whose return, unknown to the method, is $J(\theta) = 10 - (\theta - 2)^2$. Ten draws from the normal distribution with $\mu = 0$ and $\sigma = 2$ give

Draw θ	$\theta - 2$	$(\theta - 2)^2$	$J(\theta) = 10 - (\theta - 2)^2$
-2.1	-4.1	16.81	-6.81
0.4	-1.6	2.56	7.44
1.8	-0.2	0.04	9.96
-0.7	-2.7	7.29	2.71
3.2	1.2	1.44	8.56
2.5	0.5	0.25	9.75
-1.5	-3.5	12.25	-2.25
0.9	-1.1	1.21	8.79
4.4	2.4	5.76	4.24
1.1	-0.9	0.81	9.19

Step 1. The elite. The best fifth of ten draws are the two draws with the highest returns, $\theta = 1.8$ with 9.96 and $\theta = 2.5$ with 9.75.

Step 2. The new distribution.

$$\mu_{\text{new}} = \frac{1.8 + 2.5}{2} = 2.15$$

$$\sigma_{\text{new}}^2 = \frac{(1.8 - 2.15)^2 + (2.5 - 2.15)^2}{2} = \frac{0.1225 + 0.1225}{2} = 0.1225, \sigma_{\text{new}} = \sqrt{0.1225} = 0.35$$

One iteration moved the mean from 0 to 2.15, close to the best gain 2, and narrowed the spread from 2 to 0.35, or 0.4 with the constant 0.05 of the notebook. Part G of the lab runs further iterations.

Physics-informed learning

Engineering rarely starts from zero knowledge. Conservation laws, constitutive equations and boundary conditions are known even when parameters or source terms are not. Physics-informed neural networks add the residual of the governing differential equation, evaluated at many collocation points, to the loss of a network whose input is space or time [3]. Automatic differentiation provides the derivatives of the network output exactly. The network then fits the data where they exist and obeys the physics everywhere else, which makes it far more reliable outside the measured range than a data-only network. The same framework solves inverse problems, estimating unknown coefficients of the equation from data. Karniadakis and colleagues review the broader field of physics-informed machine learning [4]; related methods discover governing equations from data by sparse regression [17, 18]. Physics-informed networks can be hard to train for stiff or multiscale problems, and classical numerical solvers remain the reference where they apply.

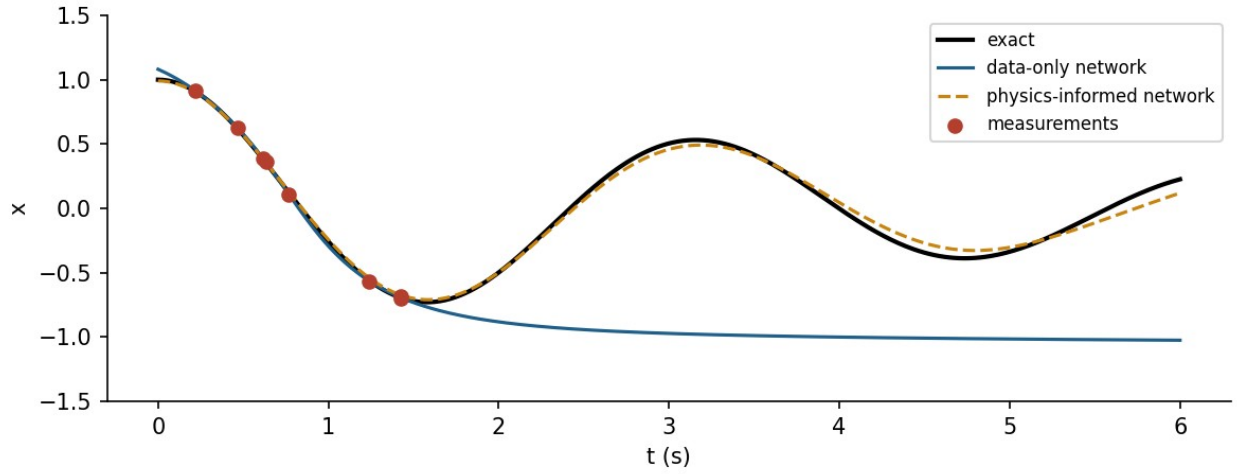


Figure 14.2. A damped oscillator: a network trained only on a few early measurements fails outside them, while a physics-informed network that also satisfies the differential equation follows the exact solution [3].

Check your understanding. What does the physics term in the loss of a physics-informed network contain?

- A. The squared error between predictions and labels only
- B. The residual of the governing equation, computed with automatic differentiation at collocation points
- C. The number of parameters of the network
- D. A random penalty

Answer: B. Minimising the residual forces the network to satisfy the equation where no data exist [3].

Formulas: The damped oscillator and the physics-informed loss

A mass m on a spring of stiffness k with a damper of coefficient c obeys $m x'' + c x' + k x = 0$, where x' and x'' are the first and second derivatives of the displacement x with respect to time. Divided by m , the equation takes the standard form

$$x'' + 2\zeta\omega x' + \omega^2 x = 0, \quad \omega = \sqrt{k/m}, \quad \zeta = \frac{c}{2\sqrt{km}}$$

with the natural angular frequency ω and the damping ratio ζ . Inserting $x = e^{st}$ gives the characteristic equation $s^2 + 2\zeta\omega s + \omega^2 = 0$. For $\zeta < 1$ its roots are $s = -\zeta\omega \pm i\omega_d$, with the imaginary unit i and the damped angular frequency $\omega_d = \sqrt{\omega^2 - (\zeta\omega)^2} = \omega\sqrt{1 - \zeta^2}$. They yield the real solutions $e^{-\zeta\omega t} \cos \omega_d t$ and $e^{-\zeta\omega t} \sin \omega_d t$, oscillations that decay with the envelope $e^{-\zeta\omega t}$, and every combination of them solves the equation. Released from $x(0) = x_0$ at rest, $x'(0) = 0$, the mass follows

$$x(t) = x_0 e^{-\zeta\omega t} \left(\cos \omega_d t + \frac{\zeta\omega}{\omega_d} \sin \omega_d t \right)$$

A physics-informed network $\hat{x}(t)$ is trained on the sum of a data loss over the N_d measurements (t_i, x_i) and a physics loss over N_c collocation points t_j , at which no measurement is needed [3]:

$$L = \frac{1}{N_d} \sum_{i=1}^{N_d} (\hat{x}(t_i) - x_i)^2 + \lambda \frac{1}{N_c} \sum_{j=1}^{N_c} (\hat{x}''(t_j) + 2\zeta\omega \hat{x}'(t_j) + \omega^2 \hat{x}(t_j))^2$$

The bracket of the second sum is the residual of the equation, which is zero for every exact solution. Automatic differentiation gives the derivatives of the network output exactly, the weight λ balances the two terms, and the initial conditions can enter as further terms of the same kind, as in the notebook [4].

Worked example: Residuals of two candidate solutions

The oscillator of the notebook obeys $x'' + 0.4x' + 4x = 0$ with $x(0) = 1$ and $x'(0) = 0$, and two measurements are $x(0.25) = 0.885$ and $x(0.5) = 0.567$.

Step 1. Frequency and damping. Comparing with the standard form gives $\omega^2 = 4$, so $\omega = 2$, and $2\zeta\omega = 0.4$, so $\zeta = 0.4/(2 \times 2) = 0.1$ and $\zeta\omega = 0.2$. Then

$$\omega_d = \sqrt{4 - 0.2^2} = \sqrt{4 - 0.04} = \sqrt{3.96} = 1.989975, \quad \frac{\zeta\omega}{\omega_d} = \frac{0.2}{1.989975} = 0.100504$$

Step 2. The exact solution at the measurement times. With

$$x(t) = e^{-0.2t} (\cos 1.989975t + 0.100504 \sin 1.989975t):$$

$$x(0.25) = e^{-0.05} (\cos 0.497494 + 0.100504 \sin 0.497494)$$

$$x(0.25) = 0.951229 \times (0.878781 + 0.100504 \times 0.477225)$$

$$x(0.25) = 0.951229 \times (0.878781 + 0.047963) = 0.951229 \times 0.926744 = 0.881546$$

$$x(0.5) = e^{-0.1} (\cos 0.994988 + 0.100504 \sin 0.994988)$$

$$x(0.5) = 0.904837 \times (0.544513 + 0.100504 \times 0.838752)$$

$$x(0.5) = 0.904837 \times (0.544513 + 0.084298) = 0.904837 \times 0.628811 = 0.568971$$

The exact solution combines solutions of the equation, so its residual is 0 at every point and its physics loss is 0.

Step 3. An undamped candidate. The candidate $x_A(t) = \cos 2t$ starts at 1 at rest, like the true motion, but ignores the damper. Its derivatives are $x_A' = -2 \sin 2t$ and $x_A'' = -4 \cos 2t$, so its residual is

$$r_A = -4 \cos 2t + 0.4 \times (-2 \sin 2t) + 4 \cos 2t = -0.8 \sin 2t$$

At four collocation points:

t	$\sin 2t$	$r_A = -0.8 \sin 2t$	r_A^2
0.5	$\sin 1 = 0.841471$	-0.673177	0.453167
1	$\sin 2 = 0.909297$	-0.727438	0.529166
1.5	$\sin 3 = 0.14112$	-0.112896	0.012746
2	$\sin 4 = -0.756802$	0.605442	0.36656

$$L_{\text{phys}} = \frac{0.453167 + 0.529166 + 0.012746 + 0.36656}{4} = \frac{1.361639}{4} = 0.34041$$

Step 4. Data and total losses. At the measurement times, $x_A(0.25) = \cos 0.5 = 0.877583$ and $x_A(0.5) = \cos 1 = 0.540302$. The data losses are

$$L_{\text{data}, A} = \frac{(0.877583 - 0.885)^2 + (0.540302 - 0.567)^2}{2} = \frac{(-0.007417)^2 + (-0.026698)^2}{2}$$

$$L_{\text{data},A} = \frac{0.000055 + 0.000713}{2} = 0.000384$$

$$L_{\text{data},\text{exact}} = \frac{(0.881546 - 0.885)^2 + (0.568971 - 0.567)^2}{2} = \frac{(-0.003454)^2 + 0.001971^2}{2}$$

$$L_{\text{data},\text{exact}} = \frac{0.000012 + 0.000004}{2} = 0.000008$$

With $\lambda = 1$, the total loss of the undamped candidate is $0.000384 + 0.34041 = 0.340794$, while that of the exact solution is 0.000008 . The data alone separate the candidates only weakly, since both stay within 0.03 of the measurements, while the physics term exposes the undamped candidate at once; a network trained on the data alone has no reason to prefer the damped motion beyond the measured range. Part H of the [interactive lab](#) evaluates these losses for other candidates, data and weights.

Digital twins

Grieves and Vickers described a digital twin as a virtual representation of a physical product that is linked to it by data throughout its life, so that behaviour can be predicted and problems anticipated [5]. Tao and colleagues surveyed its industrial use in design, production and maintenance [6]. A twin combines a model of the asset, a data connection that keeps the model's state and parameters current, and services such as monitoring, forecasting, what-if analysis and controller testing. System identification, estimating model parameters from measured inputs and outputs, is its simplest learning component; the notebook identifies the thermal parameters of the room of Week 1 from noisy data and uses the resulting model to forecast and to test a controller. The methods of the whole course appear in twins: rules and fuzzy logic for decisions, optimisation for calibration, machine learning for components without physical models, anomaly detection for monitoring and reinforcement learning for control.

Formulas: Identifying a model by least squares

The room of Week 1 changes in a time step Δt by heat loss to the outside and by the heater:

$$T_{k+1} = T_k + a(T_{\text{out}} - T_k) + b u_k, a = \frac{\Delta t}{\tau}, b = r \Delta t$$

where τ is the time constant of the room, r the heating rate in $^{\circ}\text{C}$ per minute and u_k is 1 while the heater is on. With the change $y_k = T_{k+1} - T_k$ and the inputs $x_{k1} = T_{\text{out}} - T_k$ and $x_{k2} = u_k$, the model is linear in the unknown parameters, $y_k = a x_{k1} + b x_{k2} + e_k$, where e_k collects noise and model error. Least squares chooses a and b that minimise $\sum_k e_k^2$ [18]. Setting the derivatives of this sum with respect to a and b to zero gives the normal equations

$$S_{11}a + S_{12}b = S_{1y}, S_{12}a + S_{22}b = S_{2y}$$

with the sums $S_{11} = \sum_k x_{k1}^2$, $S_{12} = \sum_k x_{k1} x_{k2}$, $S_{22} = \sum_k x_{k2}^2$, $S_{1y} = \sum_k x_{k1} y_k$ and $S_{2y} = \sum_k x_{k2} y_k$. Cramer's rule solves this system of two equations:

$$a = \frac{S_{1y} S_{22} - S_{12} S_{2y}}{S_{11} S_{22} - S_{12}^2}, b = \frac{S_{11} S_{2y} - S_{12} S_{1y}}{S_{11} S_{22} - S_{12}^2}$$

The physical parameters follow as $\tau = \Delta t / a$ and $r = b / \Delta t$. With the heater on and a constant outside temperature, the room settles where the loss balances the heating, $a(T_{\text{out}} - T) + b = 0$, at $T_{\text{eq}} = T_{\text{out}} + b/a$. The identified model is the core of a simple twin: It forecasts the room from its current state, the differences between forecasts and measurements reveal faults, and controllers can be tested on it before they touch the building [5, 6].

Worked example: Identifying the room from six measurements

A room was logged every $\Delta t = 10$ minutes at the outside temperature $T_{\text{out}} = 5^\circ\text{C}$:

k	$T_k (^\circ\text{C})$	u_k	$x_{k1} = 5 - T_k$	$y_k = T_{k+1} - T_k$
0	18.00	1	$5 - 18.00 = -13$	$18.37 - 18.00 = 0.37$
1	18.37	1	$5 - 18.37 = -13.37$	$18.66 - 18.37 = 0.29$
2	18.66	0	$5 - 18.66 = -13.66$	$18.01 - 18.66 = -0.65$
3	18.01	1	$5 - 18.01 = -13.01$	$18.33 - 18.01 = 0.32$
4	18.33	0	$5 - 18.33 = -13.33$	$17.70 - 18.33 = -0.63$
5	17.70			

Step 1. The sums. The squares and products of each step are

k	x_{k1}^2	$x_{k1} y_k$
0	$(-13)^2 = 169$	$-13 \times 0.37 = -4.81$
1	$(-13.37)^2 = 178.7569$	$-13.37 \times 0.29 = -3.8773$
2	$(-13.66)^2 = 186.5956$	$-13.66 \times (-0.65) = 8.879$
3	$(-13.01)^2 = 169.2601$	$-13.01 \times 0.32 = -4.1632$
4	$(-13.33)^2 = 177.6889$	$-13.33 \times (-0.63) = 8.3979$

$$S_{11} = 169 + 178.7569 + 186.5956 + 169.2601 + 177.6889 = 881.3015$$

$$S_{1y} = -4.81 - 3.8773 + 8.879 - 4.1632 + 8.3979 = 4.4264$$

The heater was on in steps 0, 1 and 3, so the sums with $x_{k2} = u_k$ collect only these steps:

$$S_{12} = -13 - 13.37 - 13.01 = -39.38, S_{22} = 1 + 1 + 1 = 3, S_{2y} = 0.37 + 0.29 + 0.32 = 0.98$$

Step 2. The parameters. The common denominator is

$$S_{11} S_{22} - S_{12}^2 = 881.3015 \times 3 - (-39.38)^2 = 2643.9045 - 1550.7844 = 1093.1201$$

and the two numerators give

$$a = \frac{4.4264 \times 3 - (-39.38) \times 0.98}{1093.1201} = \frac{13.2792 - (-38.5924)}{1093.1201} = \frac{51.8716}{1093.1201} = 0.047453$$

$$b = \frac{881.3015 \times 0.98 - (-39.38) \times 4.4264}{1093.1201} = \frac{863.6755 - (-174.3116)}{1093.1201} = \frac{1037.9871}{1093.1201} = 0.949564$$

Step 3. Physical parameters. The time constant is $\tau = 10/0.047453 = 210.73$ minutes and the heating rate $r = 0.949564/10 = 0.094956^\circ\text{C}$ per minute. The data were simulated with $\tau = 200$ minutes and $r = 0.1^\circ\text{C}$ per minute and carry sensor errors of about 0.02°C , so six noisy measurements recover both parameters with errors of about 5%.

Step 4. Using the twin. With the heater on in the next step, the twin forecasts

$$T_6 = 17.70 + 0.047453 \times (5 - 17.70) + 0.949564 \times 1$$

$$T_6 = 17.70 + 0.047453 \times (-12.7) + 0.949564$$

$$T_6 = 17.70 + (-0.602653) + 0.949564 = 18.046911$$

and with the heater on for good, the room would settle at the temperature

$$T_{eq} = 5 + \frac{0.949564}{0.047453} = 5 + 20.0106 = 25.0106$$

in °C, close to the 25 °C of the simulated room. A measurement far from the forecast would point to an open window, a failed heater or a faulty sensor. Part I of the [interactive lab](#) repeats the identification with more data and noise and shows how the estimates improve.

Responsible engineering of AI systems

Engineering codes of ethics require practitioners to hold public safety paramount, and they apply to systems that include learned components. Earlier work on machine ethics and AI safety argued that safety must be designed in rather than added after deployment [19]. Regulation now makes parts of this explicit. The European Union's Artificial Intelligence Act, Regulation (EU) 2024/1689, follows a risk-based approach: A few practices are prohibited, such as social scoring and the inference of emotions in the workplace and in education except for medical or safety reasons; high-risk systems, which include safety components in the management of critical infrastructure such as water, gas, heating and electricity supply, and systems used in employment and education, must meet requirements on risk management, data governance, documentation, human oversight, accuracy and robustness; and systems such as chatbots and generators of synthetic content carry transparency obligations [7].

Frameworks turn principles into practice. The NIST AI Risk Management Framework organises activities into four functions: Govern establishes policies, roles and accountability; Map establishes the context and identifies risks; Measure analyses and tracks risks with appropriate metrics; and Manage prioritises and acts on them [8]. ISO/IEC 42001 specifies a management system for organisations that develop or use AI, in the style of other management system standards [9]. Türkiye's national strategy places trustworthy and responsible AI among its priorities [20]. The energy and resource costs of training and running models are part of the same responsibility [21].

Check your understanding. Under the EU AI Act, which category most likely applies to an AI system that acts as a safety component in the management of a city's water supply network?

- A. Prohibited practice
- B. High-risk system
- C. No obligations at all
- D. Transparency obligation only

Answer: B. Safety components in the management and operation of critical infrastructure, including water supply, are listed as high-risk [7].

Formulas: How many tests support a failure rate

The Measure function of the NIST framework asks for metrics whose uncertainty is known [8]. A frequent question is how many tests without failure support a claimed failure rate of a system, for example of a vision inspection that must miss fewer than 1% of the defects. If every test fails with the probability p , independently of the others, all n tests pass with the probability

$$P(\text{no failure in } n \text{ tests}) = (1 - p)^n$$

The claim that p is below p_0 is supported with the confidence $1 - \alpha$ if a system whose rate is exactly p_0 would pass all n tests with a probability of at most α :

$$(1 - p_0)^n \leq \alpha, \text{ that is } n \geq \frac{\ln \alpha}{\ln(1 - p_0)}$$

Dividing by $\ln(1 - p_0)$, which is negative, reverses the inequality. After n tests without failure, the same reasoning gives the upper confidence bound $p_u = 1 - \alpha^{1/n}$ of the failure rate. For small rates, $\ln(1 - p_0)$ is close to $-p_0$, and with $\alpha = 0.05$, $\ln 0.05$ is close to -3 , which gives the rule of three: About $3/p_0$ tests are needed, and n passed tests bound the rate at about $3/n$. The tests must represent the conditions of use, since a shift between test and operating conditions breaks the argument [16].

Worked example: Demonstrating a miss rate below 1%

Step 1. The number of tests. With $p_0 = 0.01$ and $\alpha = 0.05$:

$$n \geq \frac{\ln 0.05}{\ln 0.99} = \frac{-2.995732}{-0.0100503} = 298.0739$$

so 299 defective parts must all be detected, close to the $3/0.01 = 300$ of the rule of three.

Step 2. A test campaign of 500 parts. If the inspection detects all 500 defective parts, the upper bound of its miss rate is

$$p_u = 1 - 0.05^{1/500} = 1 - e^{\ln 0.05/500} = 1 - e^{-2.995732/500}$$

$$p_u = 1 - e^{-0.005991} = 1 - 0.994027 = 0.005973$$

about 0.6%, again close to $3/500 = 0.006$.

Step 3. What the campaign rules out. A system that misses 1% of the defects would pass all 500 tests with the probability

$$0.99^{500} = e^{500 \ln 0.99} = e^{500 \times (-0.0100503)} = e^{-5.02515} = 0.006571$$

so such a system would almost never pass the campaign. The bound covers only the defects and conditions that the 500 parts represent: Parts from a new supplier or a different lighting need tests of their own.

The course in retrospect

The fourteen weeks form one toolbox. Search plans routes and sequences; rules encode standards; fuzzy logic turns words into control; evolutionary and swarm algorithms optimise designs; machine learning predicts, classifies and detects anomalies; deep learning perceives images and sequences; language models work with text and code; and reinforcement learning, physics-informed learning and digital twins bring learning into operation. The engineering skill lies in choosing the simplest tool that meets the requirement, validating it honestly, and knowing its limits. The final project, described in the exams folder of the repository, asks for exactly this on a problem of the student's own field.

Python step 14: Environments, learning and organised code

Functions with clear contracts

Programs grow, and the code of a final project is read by others and by its author months later. Small functions with descriptive names, type hints and docstrings make that reading easy. Type hints such as `temp_C: float` state what a parameter should be and what the function returns. Python does not enforce them, but editors and readers rely on them. The room model of Week 1 becomes a function with default parameters.

```
def room_step(temp_C: float, outside_C: float, heater: int,
              tau_min: float = 240.0, rate_C_per_min: float = 0.12,
              dt_min: float = 5.0) -> float:
    """Room temperature after one time step of the first-order model of
    Week 1."""
```

```

    return temp_C + (outside_C - temp_C) * dt_min / tau_min +
rate_C_per_min * dt_min * heater

```

```

print(room_step(20.0, 0.0, 1))

```

Output

```

20.183333333333334

```

Parameters in one place

A dataclass is a class that mainly stores values. The decorator `@dataclass` writes the `__init__` method automatically from the annotated fields, and `replace` creates a modified copy. Keeping all parameters of a model in one object avoids numbers scattered through the code and documents each variant of an experiment.

```

from dataclasses import dataclass, replace

```

```

@dataclass
class RoomParams:
    tau_min: float = 240.0
    rate_C_per_min: float = 0.12
    dt_min: float = 5.0

```

```

base = RoomParams()
insulated = replace(base, tau_min=400.0)
print(base)
print(insulated.tau_min, base.tau_min)

```

Output

```

RoomParams(tau_min=240.0, rate_C_per_min=0.12, dt_min=5.0)
400.0 240.0

```

A shared interface

Gymnasium environments share one interface: `reset` returns the first observation, and `step` returns the next observation, the reward, two flags for the end of an episode and a dictionary of extra information [11]. A simulation written with the same interface works with any agent that works with Gymnasium. The class below wraps the heating task of this week.

```

class RoomEnv:
    """Heating task with the reset and step interface of Gymnasium
    environments."""

    def __init__(self, params: RoomParams, outside_C: float = -3.5,
target_C: float = 21.0, horizon: int = 288):
        self.p, self.outside, self.target, self.horizon = params,
outside_C, target_C, horizon

    def reset(self, temp_C: float = 18.0):
        self.temp, self.k = temp_C, 0
        return self.temp, {}

    def step(self, action: int):
        p = self.p

```

```

        self.temp = room_step(self.temp, self.outside, action,
p.tau_min, p.rate_C_per_min, p.dt_min)
        self.k += 1
        reward = -abs(self.temp - self.target)
        return self.temp, reward, False, self.k >= self.horizon, {}

env = RoomEnv(base)
obs, info = env.reset()
total = 0.0
for _ in range(12):
    obs, reward, terminated, truncated, info = env.step(1 if obs < 21
else 0)
    total += reward
print(f"temperature {obs:.2f} °C, return {total:.2f}")

```

Output

temperature 19.63 °C, return -25.00

Learning to act with tabular Q-learning

Q-learning estimates the value $Q(s, a)$ of taking action a in state s and acting well afterwards [1]. After every step the estimate moves towards the observed reward plus the discounted value of the best action in the next state: $Q(s, a) \leftarrow \gamma (r + \max_{a'} Q(s', a') - Q(s, a))$. A table needs discrete states, so `np.digitize` maps the temperature to one of seven bands. The agent explores with a small probability `epsilon` and otherwise takes the action with the highest estimate. The environment class above serves unchanged.

```

import numpy as np

bands = np.array([19.0, 20.0, 20.5, 21.0, 21.5, 22.0]) # band edges
in °C: seven states

def state_of(temp_C: float) -> int:
    return int(np.digitize(temp_C, bands))

Q = np.zeros((len(bands) + 1, 2)) # one row
per state, one column per action
agent_rng = np.random.default_rng(14)
lr, gamma, eps = 0.1, 0.95, 0.1
env_q = RoomEnv(base, horizon=96) # episodes
of eight hours
for episode in range(300):
    obs, _ = env_q.reset(temp_C=float(agent_rng.uniform(17.0, 23.0)))
    s, done = state_of(obs), False
    while not done:
        a = int(agent_rng.integers(2)) if agent_rng.random() < eps else
int(Q[s].argmax())
        obs, r, terminated, truncated, _ = env_q.step(a)
        s2 = state_of(obs)
        Q[s, a] += lr * (r + gamma * Q[s2].max() - Q[s, a])
        s, done = s2, terminated or truncated
print(np.round(Q, 1))
print("heater on in states:", np.flatnonzero(Q.argmax(axis=1) == 1))

```

Output

```
[[-22.7 -20.1]
 [-13.6  -9.6]
 [-10.8  -5.5]
 [ -5.9  -4.6]
 [ -4.9  -5. ]
 [ -4.9  -5.1]
 [ -5.7  -6. ]]
heater on in states: [0 1 2 3]
```

The learned policy switches the heater on in the cold bands and off in the warm ones, a thermostat rule discovered from rewards alone. The deep reinforcement learning of this week replaces the table by a neural network when the state has many continuous variables.

Physics in the loss

Physics-informed learning adds the residual of a governing equation to the loss of a model. The room obeys $dT/dt = (T_{out} - T) / \tau + r u$. For a candidate temperature curve $T(t)$, automatic differentiation provides dT/dt at many time points, and the mean squared residual measures how strongly the curve violates the physics. Summing T before differentiating works because every value of T depends only on its own time point. The exact solution of the unheated room gives a residual of practically zero, and a straight line does not; a physics-informed network minimises such a residual together with the error on measured data.

```
import torch

t = torch.linspace(0.0, 240.0, 25, requires_grad=True)      # minutes
T_out, T0, tau = -3.5, 18.0, 240.0

def physics_loss(T, u=0.0, rate=0.12):
    """Mean squared residual of  $dT/dt = (T_{out} - T) / \tau + \text{rate } u$  at
    the points  $t$ ."""
    dTdt, = torch.autograd.grad(T.sum(), t, create_graph=True)
    return ((dTdt - (T_out - T) / tau - rate * u) ** 2).mean()

exact = T_out + (T0 - T_out) * torch.exp(-t / tau)
line = T0 - 0.05 * t
print(f"exact solution: {physics_loss(exact).item():.1e}, straight
line: {physics_loss(line).item():.1e}")
```

Output

```
exact solution: 0.0e+00, straight line: 4.4e-04
```

Tests

A test is a small piece of code that checks an expected property. The statement `assert condition` does nothing when the condition is true and raises an `AssertionError` otherwise. Physical expectations make good tests: With no temperature difference and the heater off, the room must not change, and heating must raise the temperature. Running such tests after every change catches mistakes early.

```
def test_room_step():
    assert abs(room_step(15.0, 15.0, 0) - 15.0) < 1e-12      # no
    difference, no heating
    assert room_step(15.0, 15.0, 1) > 15.0                  #
```

```

heating raises the temperature
    assert abs(room_step(20.0, 0.0, 0) - (20.0 - 20.0 * 5 / 240)) < 1e-
12
    print("all tests passed")

```

```
test_room_step()
```

Output

all tests passed

Modules and records of runs

Functions used in several notebooks belong in a module, a .py file that `import` loads. In a notebook, the first line `%%writefile` saves the rest of a cell as a file. Results should be stored together with the information needed to reproduce them: The versions of Python and of the main libraries, the parameters and the random seeds. `json` writes such a record in a readable form.

```

%%writefile roomtools.py
"""Helpers for the heating examples of the course."""

def room_step(temp_C, outside_C, heater, tau_min=240.0,
rate_C_per_min=0.12, dt_min=5.0):
    """Room temperature after one time step of the first-order
model."""
    return temp_C + (outside_C - temp_C) * dt_min / tau_min +
rate_C_per_min * dt_min * heater

```

Output

Writing roomtools.py

```

import json
import sys
import numpy as np
import roomtools

temps = [18.0]
for _ in range(288):
    temps.append(roomtools.room_step(temps[-1], -3.5, 1 if temps[-1] <
21 else 0))
record = {"python": sys.version.split()[0], "numpy": np.__version__,
"params": vars(base), "final_temp_C": round(temps[-1], 3),
"steps": len(temps) - 1}
with open("run_record.json", "w", encoding="utf-8") as f:
    json.dump(record, f, indent=1)
print(json.dumps(record))

```

Output

```

{"python": "3.12.3", "numpy": "2.4.4", "params": {"tau_min": 240.0,
"rate_C_per_min": 0.12, "dt_min": 5.0}, "final_temp_C": 21.049,
"steps": 288}

```

A final project can follow the same habits with a simple layout: A README that states the problem and how to run the code, a script or notebook that obtains the data, notebooks for exploration, a module for shared functions, and a results folder with figures and run records.

Check your understanding. What happens when the line `assert room_step(15.0, 15.0, 1) < 15.0` runs?

- A. It prints False and continues
- B. It raises an `AssertionError`, because heating raises the temperature
- C. Nothing, because `assert` statements are only comments
- D. It returns 15.0

Answer: B. The condition is false, so `assert` stops with an `AssertionError`, which is how a failing test announces itself.

Discipline challenges

The last week connects learning with action and responsibility. Choose a row and sketch the agent, the physics or the twin, together with its main risk.

Department	Challenge
Electrical and Electronics Engineering	Reinforcement learning for battery charging or demand response in a simulated microgrid, with safety limits.
Mechanical Engineering and Automotive Engineering	A digital twin of a pump or a vehicle subsystem that predicts temperatures and detects drift.
Civil Engineering	A physics-informed network for beam deflection or one-dimensional consolidation compared with the analytical solution.
Chemical Engineering and Chemistry	A physics-informed estimate of a reaction rate constant from sparse concentration data.
Environmental Engineering	Q-learning for aeration control in a simplified wastewater model with an energy penalty.
Industrial Engineering	Reinforcement learning for dispatching in a small simulated job shop compared with the rules of Week 3.
Mining Engineering	A digital twin of mine ventilation that tests fan schedules before they are applied underground.
Geophysical Engineering and Earth Sciences Engineering	Physics-informed interpolation of groundwater heads that respects Darcy flow.
Food Engineering and Biology	A twin of a drying or fermentation process identified from batch data.
Physics, Mathematics and Statistics	Discover the equation of a damped oscillator from data by sparse regression and compare with the physics-informed network [17].
Computer Engineering and Textile Engineering	An EU AI Act and NIST AI RMF assessment of an AI system your department might deploy, such as automated fabric grading or code review.

Weekly task

Prepare the final project for submission. Run its notebook in a fresh runtime from the first cell to the last, record the versions of Python and the main libraries, the parameters and the random seeds in a run record, and check the report against the template and the evaluation criteria of the final project page. Then write about 500 words of reflection on the course: Which technique family suited the problem of the project, what the baselines showed, and which risks and responsibilities would remain if the system were deployed. Cite at least three works from this week's references and three from earlier weeks [3, 7, 8].

Research and report assignment (optional)

Trustworthy learning systems in engineering. Review how reinforcement learning, physics-informed learning or digital twins are validated before deployment in one engineering domain, and relate the practices you find to the requirements for high-risk systems in the EU AI Act and to the functions of the NIST AI RMF [4, 6, 7, 8, 15].

Both assignments are optional. During an active semester, the weekly task can be sent together with the learning log exported from the interactive lab, and the research report on its own, to utkukose@sdu.edu.tr or utkukose@gmail.com. They carry no separate weight, but the instructor may take them into account as a discretionary adjustment of the midterm component of the grade. The assessment section of the syllabus gives the details, and research reports follow the report template of the course repository.

References

- [1] Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.
- [2] Watkins, C. J. C. H., & Dayan, P. (1992). Q-learning. *Machine Learning*, 8(3-4), 279-292. <https://doi.org/10.1007/BF00992698>
- [3] Raissi, M., Perdikaris, P., & Karniadakis, G. E. (2019). Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378, 686-707. <https://doi.org/10.1016/j.jcp.2018.10.045>
- [4] Karniadakis, G. E., Kevrekidis, I. G., Lu, L., Perdikaris, P., Wang, S., & Yang, L. (2021). Physics-informed machine learning. *Nature Reviews Physics*, 3(6), 422-440. <https://doi.org/10.1038/s42254-021-00314-5>
- [5] Grieves, M., & Vickers, J. (2017). Digital twin: Mitigating unpredictable, undesirable emergent behavior in complex systems. In *Transdisciplinary Perspectives on Complex Systems (F.-J. Kahlen, S. Flumerfelt, & A. Alves, Eds.)* (pp. 85-113). Springer. https://doi.org/10.1007/978-3-319-38756-7_4
- [6] Tao, F., Zhang, H., Liu, A., & Nee, A. Y. C. (2019). Digital twin in industry: State-of-the-art. *IEEE Transactions on Industrial Informatics*, 15(4), 2405-2415. <https://doi.org/10.1109/TII.2018.2873186>
- [7] European Parliament and Council of the European Union (2024). *Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act)*. Official Journal of the European Union, L series, 12 July 2024. <https://eur-lex.europa.eu/eli/reg/2024/1689/oj>
- [8] National Institute of Standards and Technology (2023). *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*, NIST AI 100-1. NIST. <https://doi.org/10.6028/NIST.AI.100-1>
- [9] International Organization for Standardization (2023). *ISO/IEC 42001:2023 Information technology, Artificial intelligence, Management system*. ISO.
- [10] Python Software Foundation (2026). *The Python Tutorial (Python 3 documentation)*. <https://docs.python.org/3/tutorial/>
- [11] Towers, M., Kwiatkowski, A., Terry, J., et al. (2024). Gymnasium: A standard interface for reinforcement learning environments. arXiv preprint arXiv:2407.17032. <https://arxiv.org/abs/2407.17032>
- [12] Mnih, V., Kavukcuoglu, K., Silver, D., et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540), 529-533. <https://doi.org/10.1038/nature14236>
- [13] Barto, A. G., Sutton, R. S., & Anderson, C. W. (1983). Neuronlike adaptive elements that can solve difficult learning control problems. *IEEE Transactions on Systems, Man, and Cybernetics*, SMC-13(5), 834-846. <https://doi.org/10.1109/TSMC.1983.6313077>
- [14] Kober, J., Bagnell, J. A., & Peters, J. (2013). Reinforcement learning in robotics: A survey. *The International Journal of Robotics Research*, 32(11), 1238-1274. <https://doi.org/10.1177/0278364913495721>

- [15] Degraeve, J., Felici, F., Buchli, J., et al. (2022). Magnetic control of tokamak plasmas through deep reinforcement learning. *Nature*, 602(7897), 414-419. <https://doi.org/10.1038/s41586-021-04301-9>
- [16] Amodei, D., Olah, C., Steinhardt, J., Christiano, P., Schulman, J., & Mané, D. (2016). Concrete problems in AI safety. arXiv preprint arXiv:1606.06565. <https://arxiv.org/abs/1606.06565>
- [17] Brunton, S. L., Proctor, J. L., & Kutz, J. N. (2016). Discovering governing equations from data by sparse identification of nonlinear dynamical systems. *Proceedings of the National Academy of Sciences*, 113(15), 3932-3937. <https://doi.org/10.1073/pnas.1517384113>
- [18] Brunton, S. L., & Kutz, J. N. (2022). *Data-Driven Science and Engineering: Machine Learning, Dynamical Systems, and Control* (2nd ed.). Cambridge University Press. <https://doi.org/10.1017/9781009089517>
- [19] Kose, U. (2018). Are we safe enough in the future of artificial intelligence? A discussion on machine ethics and artificial intelligence safety. *BRAIN. Broad Research in Artificial Intelligence and Neuroscience*, 9(2), 184-197.
- [20] Presidency of the Republic of Türkiye Digital Transformation Office, & Ministry of Industry and Technology (2021). *National Artificial Intelligence Strategy 2021-2025 (Ulusal Yapay Zekâ Stratejisi 2021-2025)*. Ankara. Action plan updated for 2024-2025. <https://www.cbddo.gov.tr/UYZS>
- [21] Strubell, E., Ganesh, A., & McCallum, A. (2019). Energy and policy considerations for deep learning in NLP. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics* (pp. 3645-3650). <https://doi.org/10.18653/v1/P19-1355>